

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>О7</u>	<u>Информационные системы и программная инженерия</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Разработка пользовательского интерфейса</u>	
		наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА

1

номер задания (при наличии)

«Автоматическое распараллеливание программ»

Вариант 12

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О717Б

Праудин Д.С.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ученая степень, ученое звание, должность

Лестенко Н.А.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

Санкт-Петербург

20 24 г.

СОДЕРЖАНИЕ

1 Постановка задачи	4
2 Реализация	9
3 Результаты тестирования	12
ЗАКЛЮЧЕНИЕ	15

1 Постановка задачи

На языке C написать консольную программу lab1.c, решающую один из вариантов, представленных ниже.

Общий вид программы представлен на рисунке 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
int main(int argc, char* argv[]) {
    int i, N;
    struct timeval T1, T2;
    long delta_ms;
    N = atoi(argv[1]); // N равен первому параметру командной строки
    gettimeofday(&T1, NULL); // запомнить текущее время T1
    for (i=0; i<100; i++) { // 100 экспериментов
        srand(i); // инициализировать начальное значение ГСЧ
        // Заполнить массив исходных данных размером N
        // Решить поставленную задачу, заполнить массив с результатами
        // Отсортировать массив с результатами указанным методом
    }
    gettimeofday(&T2, NULL); // запомнить текущее время T2
    delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 +
               (T2.tv_usec - T1.tv_usec) / 1000;
    printf("\nN=%d. Milliseconds passed: %ld\n", N, delta_ms);
    return 0;
}
```

Рисунок 1 - Общий вид программы

Здесь измеряются и отображаются временные переменные до и после выполнения определенного цикла. В приведенном примере он пуст, поэтому разница во времени будет нулевой. Внутри цикла нужно провести вычисления согласно вариантам в несколько этапов.

Этап 1 Generate. Сформировать массив M1 размерностью N, заполнив его с помощью функции rand_r (нельзя использовать rand) случайными вещественными числами, имеющими равномерный закон распределения в диапазоне от 1 до A (включительно). Аналогично сформировать массив M2 размерностью N/2 со случайными вещественными числами в диапазоне от A до 10*A.

Этап 2 Map. В массиве M1 к каждому элементу применить операцию из списка (на выбор):

- гиперболический синус с последующим возведением в квадрат;
- гиперболический косинус с последующим увеличением на 1;
- гиперболический тангенс с последующим уменьшением на 1;
- гиперболический котангенс корня числа;
- деление на Π с последующим возведением в третью степень;
- кубический корень после деления на число e ;
- экспонента квадратного корня (т.е. $M1[i] = \exp(\sqrt{M1[i]})$).

Затем в массиве M2 каждый элемент поочередно сложить с предыдущим (для этого понадобится копия массива M2, из которого нужно будет брать операнды), а к результату сложения применить операцию из списка на выбор (считать, что для начального элемента массива предыдущий элемент равен нулю):

- модуль синуса (т.е. $M2[i] = |\sin(M2[i] + M2[i-1])|$);
- модуль косинуса;
- модуль тангенса;
- модуль котангенса;
- натуральный логарифм модуля тангенса;
- десятичный логарифм, возведенный в степень e ;
- кубический корень после умножения на число Π ;
- квадратный корень после умножения на e .

Этап 3 Merge. В массивах M1 и M2 ко всем элементам с одинаковыми индексами попарно применить операцию из списка на выбор (результат записать в M2):

- возведение в степень (т.е. $M2[i] = M1[i]^{M2[i]}$);
- деление (т.е. $M2[i] = M1[i]/M2[i]$);
- умножение;
- выбор большего (т.е. $M2[i] = \max(M1[i], M2[i])$);
- выбор меньшего;

— модуль разности.

Этап 4 Sort. Полученный массив необходимо отсортировать методом, указанным в списке (для этого нельзя использовать библиотечные функции; можно взять реализацию в виде свободно доступного исходного кода):

В программе нельзя использовать библиотечные функции сортировки, выполнения матричных операций и расчета статистических величин.

В программе нельзя использовать библиотечные функции, отсутствующие в стандартных заголовочных файлах `stdio.h`, `stdlib.h`, `sys/time.h`, `math.h`.

Список предложенных сортировок:

- сортировка выбором (Selection sort);
- сортировка расчёской (Comb sort);
- пирамидальная сортировка (HeapSort, сортировка кучи);
- гномья сортировка (Gnome sort);
- сортировка вставками (Insertion sort);
- сортировка выбором (Selection sort).

Этап 5 Reduce. Рассчитать сумму синусов тех элементов массива M2, которые при делении на минимальный ненулевой элемент массива M2 дают четное число (при определении чётности учитывать только целую часть числа). Результатом работы программы по окончании пятого этапа должно стать одно число X, которое следует использовать для верификации программы после внесения в нее изменений (например, до и после распараллеливания итоговое число X не должно измениться в пределах погрешности). Данное число необходимо выводить на каждой итерации на этапе верификации. Значение числа X следует привести в отчете для различных значений N.

ВАЖНО! Циклов в программе несколько. Внешний цикл, приведенный на скриншоте, – это количество экспериментов. Все описанные манипуляции с массивами проводятся каждая в своем цикле: цикл заполнения массива M1, заполнение массива M2 и т.д.

Результатом выполнения программы является количество миллисекунд, за которые выполняется внешний цикл из 100 итераций. Время выполнения варьируется за счет размерности массивов N, которая передается в программу в качестве аргумента при запуске, а не количества экспериментов. Проще говоря, число 100 из внешнего цикла убирать не надо.

Скомпилировать написанную программу нужно в двух вариантах.

Последовательно — без использования автоматического распараллеливания с помощью следующей команды:

```
/home/user/gcc -O3 -Wall -Werror -lm -o lab1-seq lab1
```

В этом случае получается файл lab1-seq.

Запустить программу на выполнение нужно следующей командой:

```
./lab-seq
```

Также, используя встроенное в gcc средство автоматического распараллеливания Graphite, с помощью следующей команды:

```
//gcc -O3 -Wall -Werror -lm -floop-parallelize-all -ftree-parallelize-loops=K  
lab11.c -o lab1-par-K
```

K – параметр, отвечающий за количество параллельных потоков. Вместо него нужно подставить число. Предлагается запустить компиляцию несколько раз, меняя число K соответствующим образом:

- один;
- меньше числа физических ядер;
- равное числу физических ядер;
- больше числа физических ядер;

Например, при наличии 4 процессоров в виртуальной машине это будет 1, 2, 4, 6.

```
//gcc -O3 -Wall -Werror -lm -floop-parallelize-all -ftree-parallelize-loops=2  
lab11.c -o lab1-par-2
```

Соответственно получится четыре файла:

- lab1-par-1;
- lab1-par-2;

— lab1-par-4;

— lab1-par-6.

В результате получится одна не распараллеленная программа и четыре или более распараллеленных.

Закрывать все работающие в операционной системе прикладные программы, чтобы они не влияли на результаты последующих экспериментов. При использовании ноутбука необходимо иметь постоянное подключение к сети питания на время проведения эксперимента.

Запускать файл lab1-seq из командной строки, увеличивая значения N до значения N_1 , при котором время выполнения превысит 0.01с.

Подобным образом найти значение $N=N_2$, при котором время выполнения превысит 5с.

Используя найденные значения N_1 и N_2 , выполнить следующие эксперименты (для автоматизации проведения экспериментов можно написать скрипт):

- запускать lab1-seq для значений $N = N_1, N_1 + \Delta, N_1 + 2\Delta, N_1 + 3\Delta, \dots, N_2$ и записывать получающиеся значения времени $\text{delta_ms}(N)$ в функцию $\text{seq}(N)$;
- запускать lab1-par-K для значений $N = N_1, N_1 + \Delta, N_1 + 2\Delta, N_1 + 3\Delta, \dots, N_2$ и записывать получающиеся значения времени $\text{delta_ms}(N)$ в функцию $\text{par-K}(N)$;
- значение Δ выбрать так: $\Delta = (N_2 - N_1)/10$.

Сравнить Δ при выполнении программы последовательно и параллельно.

Представить таблицу с временем выполнения программ при разных N .

Текст программы нужно сохранить для будущих практических работ и курсового проекта.

2 Реализация

Lab1.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <sys/time.h>

// Размерность массивов
#define N1 1000
#define N2 10000
#define A 100

void generateArrays(float *M1, float *M2, int N) {
    unsigned int seed = 0;
    for (int i = 0; i < N; i++) {
        M1[i] = 1 + (float)rand_r(&seed) / RAND_MAX * (A - 1);
    }
    for (int i = 0; i < N / 2; i++) {
        M2[i] = A + (float)rand_r(&seed) / RAND_MAX * (10 * A - A);
    }
}

void mapArrays(float *M1, float *M2, int N) {
    for (int i = 0; i < N; i++) {
        M1[i] = sinh(M1[i]) * sinh(M1[i]); // пример операции для M1
    }
    float *M2_copy = (float *)malloc((N / 2) * sizeof(float));
    for (int i = 0; i < N / 2; i++) {
        M2_copy[i] = M2[i];
    }
    for (int i = 1; i < N / 2; i++) {
        M2[i] += M2_copy[i - 1];
        M2[i] = fabsf(sinf(M2[i])); // пример операции для M2
    }
    free(M2_copy);
}

void mergeArrays(float *M1, float *M2, int N) {
    for (int i = 0; i < N / 2; i++) {
```



```

        M2[i] = powf(M1[i], M2[i]); // пример операции степень
    }
}

```

```

void combSort(float *arr, int n) {
    int gap = n;
    int swapped = 1;
    while (gap != 1 || swapped == 1) {
        gap = (gap * 10) / 13;
        if (gap < 1)
            gap = 1;
        swapped = 0;
        for (int i = 0; i < n - gap; i++) {
            if (arr[i] > arr[i + gap]) {
                float temp = arr[i];
                arr[i] = arr[i + gap];
                arr[i + gap] = temp;
                swapped = 1;
            }
        }
    }
}

```

```

float reduceArray(float *M2, int N) {
    float min = M2[0];
    for (int i = 1; i < N / 2; i++) {
        if (M2[i] < min && M2[i] != 0) {
            min = M2[i];
        }
    }
    float sum = 0;
    for (int i = 0; i < N / 2; i++) {
        if ((int)(M2[i] / min) % 2 == 0) {
            sum += sinf(M2[i]);
        }
    }
    return sum;
}

```

```

int main(int argc, char *argv[]) {

```

```

if (argc < 2) {
    printf("Usage: %s <N>\n", argv[0]);
    return 1;
}

int N = atoi(argv[1]);

float *M1 = (float *)malloc(N * sizeof(float));
float *M2 = (float *)malloc((N / 2) * sizeof(float));

struct timeval T1, T2;
long delta_ms;
float result;

for (int i = 0; i < 100; i++) {
    gettimeofday(&T1, NULL);

    srand(i);
    generateArrays(M1, M2, N);
    mapArrays(M1, M2, N);
    mergeArrays(M1, M2, N);
    combSort(M2, N / 2);
    result = reduceArray(M2, N);

    gettimeofday(&T2, NULL);
    delta_ms = (T2.tv_sec - T1.tv_sec) * 1000 + (T2.tv_usec - T1.tv_usec) / 1000;

    printf("Iteration %d: X = %f, Milliseconds passed: %ld\n", i, result,
delta_ms);
}

free(M1);
free(M2);

return 0;
}

```

3 Результаты тестирования

Данная работа была выполнена на устройстве, комплектующие которого представлены в таблице 1:

Таблица 1 - Комплектующие компьютера

Процессор	11th Gen Intel(R) Core (TM) i7-11800H @ 2.30GHz 2.30 GHz
Оперативная память	16,0 ГБ (доступно: 15,7 ГБ)
Тип системы	64-разрядная операционная система, процессор x64

На данном компьютере запускается VirtualBox debian-12.5.0-amd64-netinst. Было настроено потока на ядро, изображено на рисунке 2 и была проведена проверка статуса гипертрединга на рисунке 3.

```
danya@debian: ~$ cat /proc/cpuinfo |grep 'core id'
core id      : 0
core id      : 1
core id      : 2
core id      : 3
```

Рисунок 2 – Проверка количества потоков на ядро

```
danya@debian: ~$ cat /sys/devices/system/cpu/smt/active
0
```

Рисунок 3 – Проверка статуса гипертрединга

Компиляция программы представлена на рисунке 4:

```
danya@debian: ~$ gcc -O3 -Wall -Werror -o lab1-seq lab1.c -lm
danya@debian: ~$ ./lab1-seq
Usage: ./lab1-seq <N>
```

Рисунок 4 – Компиляция программы

Сделаем автоматическое распараллеливание с помощью Graphite, данное действие изображено на рисунке 5:

```
danya@debian: gcc -O3 -Wall -Werror -floop-parallelize-all -ftree-parallelize-loops=1
-o lab1-par-1 lab1.c -lm
gcc -O3 -Wall -Werror -floop-parallelize-all -ftree-parallelize-loops=2 -o lab1-par-2 l
ab1.c -lm
gcc -O3 -Wall -Werror -floop-parallelize-all -ftree-parallelize-loops=4 -o lab1-par-4 l
ab1.c -lm
gcc -O3 -Wall -Werror -floop-parallelize-all -ftree-parallelize-loops=6 -o lab1-par-6 l
ab1.c -lm
danya@debian:$ ./lab1-par-1
./lab1-par-2
./lab1-par-4
./lab1-par-6
Usage: ./lab1-par-1 <N>
Usage: ./lab1-par-2 <N>
Usage: ./lab1-par-4 <N>
Usage: ./lab1-par-6 <N>
danya@debian:$
```

Рисунок 5 – Автоматическое распараллеливание с помощью Graphite

Результат работы lab1-seq при N = 2400 изображен на рисунке 6:

```
Iteration 93: X = -105.903580
Iteration 94: X = -105.903580
Iteration 95: X = -105.903580
Iteration 96: X = -105.903580
Iteration 97: X = -105.903580
Iteration 98: X = -105.903580
Iteration 99: X = -105.903580
N=2400. Milliseconds passed: 9.000000
```

Рисунок 6 – Результат работы lab1-seq

Результаты работы lab1-par-1-6 при N = 2400 изображены на рисунках 7-10:

```
Iteration 95: X = 0.328418
Iteration 96: X = 0.328418
Iteration 97: X = 0.328418
Iteration 98: X = 0.328418
Iteration 99: X = 0.328418
N=2400, Threads=1. Milliseconds passed: 13.000000
```

Рисунок 7 – Результат работы lab1-par-1

```
Iteration 95: X = 0.328418
Iteration 96: X = 0.328418
Iteration 97: X = 0.328418
Iteration 98: X = 0.328418
Iteration 99: X = 0.328418
N=2400, Threads=2. Milliseconds passed: 8.000000
```

Рисунок 8 – Результат работы lab1-par-2

```

Iteration 95: X = -105.903580
Iteration 96: X = -105.903580
Iteration 97: X = -105.903580
Iteration 98: X = -105.903580
Iteration 99: X = -105.903580
N=2400, Threads=4. Milliseconds passed: 9.000000

```

Рисунок 9 – Результат работы lab1-par-4

```

Iteration 95: X = 295.760468
Iteration 96: X = 295.760468
Iteration 97: X = 295.760437
Iteration 98: X = 295.760468
Iteration 99: X = 295.760468
N=2400, Threads=6. Milliseconds passed: 103.000000

```

Рисунок 10 – Результат работы lab1-par-6

Методом подбора, мы находим нужные нам значения для N1 и N2.

$N1=2400 > 0.01с.$

$N2= 560000 > 5с.$

Находим $\Delta = (N2 - N1)/10 \Rightarrow (560000-2400)/10 = 55760.$

Используя найденные значения N1 и N2, выполним следующие эксперименты, изображённые в таблице 2.

Таблица 2 – Эксперементы с разным количеством элементов

Программа	N = N1, N1+ Δ, N1+2Δ,..., N2										
N	2400	55760	113920	169680	225440	281200	336960	392720	448480	504240	560000
Lab1-seq	10	350	722	1218	1507	2082	2534	3026	3701	4298	4720
Lab1-par-1	9	350	767	1094	1606	1953	2434	2963	3554	4252	4604
Lab1-par-2	9	353	770	1139	1579	2009	2440	3057	3545	3964	4667
Lab1-par-4	12	335	724	1205	1544	2176	2512	3138	3729	3986	4604
Lab1-par-6	16	344	779	1155	1609	1981	2477	2988	3382	3975	4479

ЗАКЛЮЧЕНИЕ

Автоматическое распараллеливание с использованием Graphite показало улучшение производительности при увеличении количества потоков до определенного значения.

При дальнейшем увеличении количества потоков производительность незначительно изменилась, что может быть связано с накладными расходами на управление потоками.